HCL HACKATHON

End-to-End Capstone Project

Smart HR Workforce Analytics

From scattered HR records to clear retention answers with SQL analytics and practical GenAI

Submitted by

Vikash Kumar Singh

B.Tech, Computer Science and Engineering

Ajay Kumar Garg Engineering College, Ghaziabad

2026

1. Project Title and Team Details

Project title: Smart HR Workforce Analytics.

Team: I am working alone on this proposal. The notice allows a maximum of 2 members. I have not named a second member, because no one has been confirmed yet. If the college assigns someone before submission, I will add that name and split the work as shown in Section 14. Submission deadline: 4 November 2026. Report filename: VikashKumarSingh_SmartHRWorkforceAnalytics_Proposal.pdf. My roll number, 2300271540137, is taken from my college project records; it is not printed in my resume PDF, so I will check it against my college ID card before submission.

2. Executive Summary

Companies lose employees without warning, and hiring a replacement costs much more than keeping the person who left. HR teams keep attendance, leave, salary and satisfaction data in separate files, so they cannot see which departments, roles or work patterns are linked to people leaving. I propose Smart HR Workforce Analytics to fix that gap.

My plan is to take the IBM HR Analytics dataset from Kaggle, plus a small HR API feed, clean and check the data, and store it in PostgreSQL for employees, departments and facts. I will then use SQL to answer questions on attrition by department, overtime, satisfaction, income and tenure, and show them on a simple React dashboard. I will also add a GenAI part that explains those SQL results in plain words, answers HR policy questions from uploaded documents, and turns plain English into safe, read-only SQL.

The result I want is a working demo for HR managers, giving early warning signs and evidence for retention work. I am not claiming it can predict what any one person will do. It can only point to groups where risk looks higher in this data.

Project at a Glance

Item	In this project
Dataset	IBM HR Analytics, Kaggle: 1,470 records, 35 columns
Database	PostgreSQL: dim_employee, fact_employment, fact_satisfaction, fact_attrition, load_log
Analytics	7 SQL questions (Q1 to Q7), all from stored tables
GenAI	Explain SQL results, policy Q and A with quotes, plain English to read-only SQL
Demo	React dashboard plus a 5-minute walkthrough, from raw file to GenAI explanation

3. Real-World Problem Statement

I saw part of this problem during my internship, and it is simple to state. HR managers in mid-size companies face three problems. First, their data is scattered. Employee details sit in one file, attendance in another, and leave and salary somewhere else. No one has one clean place to look. Second, decisions come too late. A resignation often arrives as a surprise. By then, the cost of hiring, training and lost work has already started. Third, the reports they get are too general. One attrition percentage for the whole company does not tell them where the

problem is. It does not say which department, which role, or which tenure band is hit hardest. It also does not say whether overtime or low satisfaction goes along with it.

So my problem is not “make a chatbot”. It is to build one clean HR dataset that can be queried, with SQL answers that can be checked, so an HR manager can see a number, see where it came from, and act before the next resignation. Past patterns show risk groups only; they cannot prove that a named person will leave.

4. Objectives and Expected Outcomes

- O1 — One pipeline, end to end. I will build CSV plus API ingestion, cleaning, PostgreSQL storage, SQL analytics and a GenAI explanation, all working in one demo.
- O2 — At least 7 SQL answers. The questions are listed in Section 10. Each answer must come from the stored tables. I will not type numbers by hand into the dashboard.
- O3 — A dashboard a non-technical HR user can read fast. Department attrition, overtime, satisfaction, income and tenure views, aimed to be readable in under one minute.
- O4 — GenAI that stays tied to the data. It will explain SQL results in plain words, answer policy questions with the source text shown, and turn natural language into SQL that can only read data, never change it.
- O5 — A demo I can measure. Section 15 lists how I will test it. “It works” on its own is not enough for me.

What I expect at the end: a working demo, a GitHub repository, this proposal report, and a 5-minute presentation. In that presentation I will take one question all the way from the raw file to the dashboard to the GenAI explanation.

5. Industry Relevance and Proposed Innovation

Workforce analytics is normal HR work in IT services, consulting and large companies that manage large teams across clients. That is why this project fits an industry setting. It is not only a classroom problem.

I am not claiming a new algorithm here. My point is different, and I think it matters more for this project. Every GenAI answer in my system must be tied to a SQL result or to a quoted policy passage, so a manager can check it. A common risk in GenAI demos is invented numbers. I will not allow that in my design. Numbers come only from the database. The model only explains them.

The second point is about how I report risk. I will report groups, not people. For example, if a slice such as a department, early tenure and overtime showed higher attrition in this data, the system would report it as a risk group for review. I will not label a named employee as “will leave”.

My third point is personal, and it is the part an AI tool cannot copy for someone else. At TalAstra I worked on a production HRMS with employee, attendance, leave, onboarding and helpdesk modules. I have seen how an HR manager actually works: a weekly review, one screen, a risk group, and then an action. My dashboard is built for that weekly review, not for a demo screenshot.

Why this proposal is different from a typical AI-made proposal

What is usually shown	What mine will show
A model that predicts which person will leave	SQL that shows which groups are at risk. No named employee is labelled as going to leave.

What is usually shown	What mine will show
A chatbot that answers from memory	Every GenAI answer shows its SQL, its row count, and its policy quote. If any of these is missing, it says data not available.
Screenshots as proof	A re-run proof: same CSV in, same numbers out, with a load log and a pandas cross-check for every SQL answer.
Unverifiable company data	A public Kaggle dataset with a link, so any reviewer can download it and check the numbers.

6. Dataset Details and Data Source Links

Dataset	Details	Source
Primary: IBM HR Analytics Employee Attrition and Performance	1,470 employee records, 35 columns. Attrition Yes/No: 237 Yes (16.1%), 1,233 No. Main fields: Age, Department, JobRole, MonthlyIncome, MonthlyRate, JobSatisfaction, EnvironmentSatisfaction, WorkLifeBalance, OverTime, YearsAtCompany, YearsInCurrentRole, YearsWithCurrManager, TotalWorkingYears, DistanceFromHome, Education, EducationField, Gender, MaritalStatus, PerformanceRating, PercentSalaryHike. Fictional dataset made by IBM data scientists, widely used for teaching HR analytics.	https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset
Secondary feed (planned, small)	A small supplementary feed of attendance and leave records, using the same employee-number keys, built for the demo to show API ingestion. All main analytics still come from the primary dataset, so results stay repeatable. No real employee personal data is used.	Demo API built for this project (Node/Express, local) and CSV files in the repository.
Policy documents (GenAI only)	2 to 3 short HR policy texts written by me: leave, overtime, exit process, stored as text files. They are demo documents, not a real company's policies, and will be labelled as such.	Author-written, in the repository under /docs/policies/.

On data ethics: no real names, phone numbers or email addresses are used. EmployeeNumber is just a dataset ID, and the fictional nature of the dataset will be stated on the dashboard.

7. Data Ingestion Methodology (Files / APIs)

- Step 1 — File ingestion. I will download the Kaggle CSV once, note its checksum, and store it unchanged in /data/raw/. I will never edit the raw file. Every later step will read from it.
- Step 2 — API ingestion. I will build a small Node/Express demo API that serves attendance and leave data as JSON, in pages, with an API key for the demo. A Python script will call that API, log how many rows came back and what HTTP status it got, and save the response unchanged in /data/raw/api/.
- Step 3 — Load log. Every run will write the date, the source, rows received, rows loaded and any failed rows into a log table. If a run loads 0 rows, the dashboard will keep showing the last good load and will flag “data not refreshed”. It will not show empty charts and pretend that is an answer.
- Step 4 — No duplicates on re-run. If I run ingestion again, it must not double the rows. The natural key is EmployeeNumber plus source. Load date is recorded as audit information only, not as part of the key, so a re-run on another day updates the same row instead of creating a new one.

8. Data Cleaning and Transformation Methodology

I will clean the data in Python with pandas. Tenure in this project means YearsAtCompany. Income bands are a design choice I define here: below 3,000, 3,000 to 6,000, 6,000 to 10,000, and above 10,000 in monthly income units as given in the dataset.

Problem in the raw data	Rule I will use
Columns that never change	Drop EmployeeCount (always 1) and StandardHours (always 80) after writing down why. A column that never changes cannot explain why someone leaves. Over18 is also constant (Y) in this dataset, so it is kept in dim_employee for completeness but is not used as an analytical field.
Attrition written as text	Map Yes to 1 and No to 0 in a new column, and keep the original text column as well, so I can check my work later.
Messy category text	Trim extra spaces and fix the case for Department, JobRole, MaritalStatus, BusinessTravel and OverTime. Any value outside my allowed list will go to a quarantine table for review, not into the main tables.
Numbers that do not make sense	Flag rows where Age is less than 18 or more than 70, where income is negative, or where YearsAtCompany is more than TotalWorkingYears. I will not delete those rows quietly. They will go to quarantine.
Duplicate rows	Drop exact duplicate EmployeeNumber rows, keep the first one, and log how many I dropped. Dropped duplicates are counted separately in the row-count test in Section 15.

Problem in the raw data	Rule I will use
New fields I need	Tenure band (0-2, 3-5, 6-10, 10+ years, based on YearsAtCompany), income band as defined above, an overtime flag, and a satisfaction score, which is the mean of job, environment and relationship satisfaction. I will write down each new field, and no new field will hide a raw value.

9. Database Design and Storage Technologies

I chose PostgreSQL. HR data has strong links between tables — an employee, their leave, their attendance — and a relational database handles those links well. I used PostgreSQL in my TalAstra HRMS work too, so I am not picking it blind. Python with pandas will handle ingestion and cleaning. For a dataset of this size, I do not need a NoSQL store, and adding one would not make the demo better. All 35 dataset columns are accounted for below: 33 are stored, and only EmployeeCount and StandardHours are dropped, for the reasons given in Section 8.

Table	Main columns	What it is for
dim_employee	employee_number (primary key), age, gender, marital_status, education, education_field, department, job_role, job_level, over18	One row per employee. Basic details.
fact_employment	employee_number (foreign key), monthly_income, monthly_rate, daily_rate, hourly_rate, percent_salary_hike, stock_option_level, years_at_company, years_in_role, years_with_curr_manager, years_since_promotion, total_working_years, num_companies_worked, distance_from_home, business_travel, overtime	Measurable facts about the job, linked to the employee.
fact_satisfaction	employee_number (foreign key), job_satisfaction, environment_satisfaction, relationship_satisfaction, work_life_balance, job_involvement, performance_rating, training_times_last_year	Survey-style scores. Kept apart so a score change does not rewrite the job facts.
fact_attrition	employee_number (foreign key), attrition (0/1), attrition_text	The outcome table. Every analytical query joins here, so the outcome never gets mixed into the other tables.
load_log	load_id, source, load_date,	A record of every ingestion

Table	Main columns	What it is for
	rows_received, rows_loaded, rows_quarantined, duplicates_dropped	run, for checking later.

10. Proposed SQL Analytical Questions

The notice asks for at least 5 questions. I propose 7.

No.	Question	SQL idea
Q1	What is the attrition rate by department?	Group by department, average of attrition times 100. Shows where loss is concentrated.
Q2	Does overtime go along with attrition?	Attrition rate by overtime (Yes/No), first for everyone, then by department.
Q3	Which job roles lose people fastest?	Attrition rate and headcount by job_role, ordered by rate. Headcount is shown too, so a very small role is not mistaken for a trend.
Q4	How do satisfaction and work-life balance relate to leaving?	Average satisfaction scores for people who left, compared with people who stayed, split by satisfaction band (Low 1-2, Medium 3, High 4).
Q5	Is early tenure the riskiest period?	Attrition rate by tenure band (0-2, 3-5, 6-10, 10+ years, based on YearsAtCompany).
Q6	Does income level matter, apart from role?	Attrition rate by income band, with a second query that also splits by department.
Q7	Where should HR act first?	A ranked attention list: departments and roles with both a high attrition rate and a headcount of at least 50, so action goes to real scale, not noise.

Example for Q1, in simplified form:

```
SELECT department, COUNT(*) AS headcount,
ROUND(AVG(attrition)*100,1) AS attrition_pct
FROM dim_employee
JOIN fact_attrition USING (employee_number)
GROUP BY department
ORDER BY attrition_pct DESC;
```

11. Complete End-to-End System Architecture Diagram

Sources, then ingestion in Python, then cleaning in pandas, then PostgreSQL, then SQL analytics, then the React dashboard and the GenAI layer. There is no arrow from GenAI to the database, because that connection does not exist in my design.

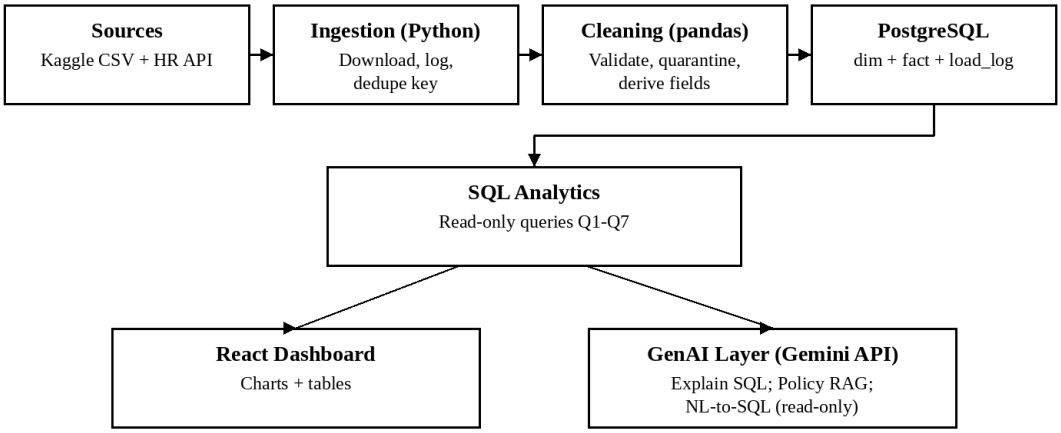


Figure 1: End-to-end system architecture. GenAI explains database results; it does not write to the database.

Guardrail for the whole diagram: GenAI explains database results. It never makes up a number, and it never writes to PostgreSQL.

12. GenAI Integration — Practical, Not Just a Chatbot

The notice says a project that is only a chatbot is not enough. In my project, if you remove GenAI, the pipeline still works. GenAI is only an explanation layer on top of checked data, in three parts.

- G1 — Explain the SQL result. The dashboard will send the SQL question, the SQL itself, and the numbers that came back to the model. The model will return a short explanation in plain words, 3 to 4 lines, plus one suggested HR action. If the numbers are missing, it must say “data not available”. It must not guess.
- G2 — Policy questions with sources shown. I will split my short HR policy texts, search them for the relevant passage, and ask the model to answer from that passage. The answer must quote the passage it came from. If the question is outside those documents, the answer must say “not found in the provided policies”.
- G3 — Plain English to SQL, read-only only. A user can type something like “attrition by department”. The model will suggest a SELECT query only. The query will run under a read-only database role, and my backend will also reject any query that contains INSERT, UPDATE, DELETE, DROP or ALTER. The SQL and the result will both be shown, so the user can check the work, and the database stays safe.

Evidence rule for every GenAI answer on screen: the SQL used, the rows counted, and the source passage will be shown below the answer. If any one of these is missing, the answer will not be shown. This is what makes the GenAI part checkable in a live demo, not just believable in a report.

13. Week-Wise Implementation Roadmap with Milestones

The proposal itself is due on 4 November 2026. The four weeks below are my implementation plan after proposal approval. I am listing them here because the notice asks for a roadmap, not because I claim this work is done.

Week	What I will do	Milestone
Week 1 — Foundations	Download the dataset and note its checksum, store the raw file, create the PostgreSQL schema, write the ingestion script for CSV and API, and set up the load log.	The database loads 1,470 rows, and a re-run produces 0 duplicates.
Week 2 — Clean and analyse	Apply the cleaning rules and quarantine, add the derived fields, write and test SQL questions Q1 to Q7, and save the result tables.	All 7 SQL answers run, and they match a manual pandas cross-check.
Week 3 — Dashboard and GenAI	Build the React dashboard with charts from the API, not hand-typed data, then add the GenAI explanation, the policy question part, and the natural-language-to-SQL part with its read-only guard.	One question flows live from file to database to chart to GenAI explanation.
Week 4 — Test and present	Run the tests in Section 15, check performance, write the README, prepare the demo script, and put together the final presentation and submission pack.	A 5-minute demo, a repository, and a final report, ready for review.

14. Proposed Technology Stack and Team Responsibilities

Layer	Technology	Why I chose it
Ingestion and cleaning	Python, pandas, requests (proposed implementation tools)	I already know Python, and pandas is the normal tool for CSV work.
Storage	PostgreSQL	Relational HR data, and I used it in my TalAstra HRMS work.
API	Node.js / Express, for the demo API and the dashboard API	This is my full-stack stack, and it keeps the demo API in one language.
Frontend	React, Tailwind CSS	I have built React apps before, and components suit a dashboard.
GenAI	Gemini API, as used in MediCare, with Groq, as used in Intellexa, as a fallback option	I have used both APIs in past projects. Keys will live in environment variables only, never in the code.
Tools	Git/GitHub, VS Code, Postman	Version control, API testing, and the code-review habit I

Layer	Technology	Why I chose it
		picked up at TalAstra.

Who does what: Ingestion, cleaning, database design and SQL analytics: me, Vikash Kumar Singh, as lead. React dashboard, GenAI integration, testing and demo script: a second member, if the college assigns one. I have not named anyone here, because no one is confirmed. If I stay alone, I will do that part too, and Week 3 will take 3 days longer. I would rather write that plainly now than invent a name.

15. Testing Strategy, Expected Results and Final Deliverables

Testing strategy

- Row-count test. Raw rows must equal cleaned rows plus quarantined rows plus duplicates dropped, on every run. If that fails, I stop and fix cleaning before I move on.
- SQL cross-check. I will compare each SQL answer with a separate pandas calculation on the CSV. Any mismatch blocks that milestone. I will not move on with a number I cannot reproduce two ways.
- Guardrail tests. The natural-language-to-SQL part must reject INSERT, UPDATE, DELETE, DROP and ALTER, and it must run under a read-only database role. The GenAI explanation must not change a number. I will check output numbers against the SQL result before anything is shown.
- Dashboard tests. Every chart value must trace back to one API response. No hard-coded chart data in the frontend.
- Performance targets. Each analytical query should return in under 2 seconds on the 1,470-row dataset on a normal laptop. An ingestion re-run should finish in under 1 minute. These are targets, not results I have measured yet, because the system is not built yet.

Expected results, stated honestly

I expect to see differences in attrition by department and role, higher attrition where overtime is marked Yes and satisfaction is lower, in this fictional dataset, and a usable early-warning view for HR. I am not claiming those patterns will hold for a real company, and I am not predicting any one person.

Final deliverables

- 1. This proposal report, as a PDF.
- 2. A GitHub repository with ingestion, cleaning, schema, SQL, API, dashboard and GenAI code.
- 3. A working local demo.
- 4. A 5-minute presentation that walks one question from the raw file to the GenAI explanation.

Limitations and risks

- The main dataset is fictional, and it has only 1,470 rows. The results do not transfer to a real workforce without real data and a proper HR review.
- Attrition in the data shows a link, not a cause. Overtime or satisfaction may travel with other factors I have not measured.
- GenAI can misread a question. My read-only guard and quoted sources lower that risk, but they do not remove it. That is why every number stays traceable to SQL.